

Transformer Toy Model Sequence Memorisation

1 Why sequence memorisation

We expect that a lot of the information stored in the weights of a transformer is memorised facts, rather than general circuits.

2 The training data

The training data are sequences of three token, one two input tokens and one output token. Given an input of tow tokens, the network is trained to predict the next token (i.e. the output token).

Hyper parameters for the data generation:

- input token vocabulary size
- output token vocabulary size

In our experiments, the input token vocabulary size is always twice the size as the output token vocabulary size.¹

2.1 Generating the training data

The code for generating the training data is not very long, and is quoted below if you prefeer to read code.

The inputs are

- `n_facts` – Number of facts.
- `input_len` – Number of input tokens. *This value is aways 2.*
- `input_vocal_size`
- `output_vocab_size`
- `seed` – Random seed. *This value is always 42.*²

The code first generates a list of every possible input combination. Then this list is shuffled, and the first `n_fact` pairs from the shuffled list are used as the inputs for the `n_fact` facts. These facts are then devided as equal as is possible among the `output_vocab_size` target labels.

[Make a colapsable box for the code below]

¹We started out the experiments, having them the same size, but the network learned all the facts to easaly, so we dubbled the input vocabulary size in order to have more possible facts.

²We used a fixed random seed to avoid some runs getting lucky and getting easier fracts, and to specifically have the same facts for trained networks and hand-coded networks. However this last aim failed becasuse torch random functions give diffrent results diffrent when run on CPU vs GPU, even with the seed is the same. However, this probably isn't significant a significant concern.

```

def generate_facts(n_facts: int, # of facts to generate,
                  input_len: int, # number of input tokens per fact
                  input_vocab_size: int, # of unique tokens in the vocabulary
                  output_vocab_size: int, # of unique targets
                  seed: int = 42
                  ) -> dict[str, torch.Tensor]:

    if n_facts > input_vocab_size ** input_len:
        raise ValueError(f"Cannot generate {n_facts} unique facts with a vocabulary of size {input_vocab_size ** input_len}")

    device = torch.tensor(0).device # respect default device
    generator = torch.Generator(device=device).manual_seed(seed)

    targets = torch.arange(n_facts) % output_vocab_size

    if input_len == 1:
        inputs = torch.randperm(input_vocab_size, generator=generator)[:n_facts].unsqueeze(1)
    elif input_len == 2:
        all_possible_inputs = torch.cartesian_prod(torch.arange(input_vocab_size), torch.arange(output_vocab_size))
        inputs = all_possible_inputs[torch.randperm(all_possible_inputs.size(0), generator=generator)[:n_facts]]
    else:
        inputs = torch.randint(0, input_vocab_size, (n_facts, input_len), generator=generator)

    sorted_indices = torch.argsort(targets)
    return {"inputs": inputs[sorted_indices], "targets": targets[sorted_indices]}

```

3 Model Architecture

The full toy model is a small one layer transformer. In addition to training the full model, we also try turning off various parts in various combination, to see what parts of the model is important for the sequence memorisation task.

The full toy model consist of:

- Token embedding
- Positional embedded
- A single full width attention head
- A MLP layer (on the last token position only since we're not trying to predict intermediate tokens)
- Two residual connections, one passed the attention, and one passed the MLP.
- Token unembedding to create the logits for the target tokens.
- Three RMSNorms, one applied to the input to the attention, one to the input to the MLP and one to the input to the unembedding.

3.1 Model variations

3.1.1 Attention

There are the variants when it comes to the attention.

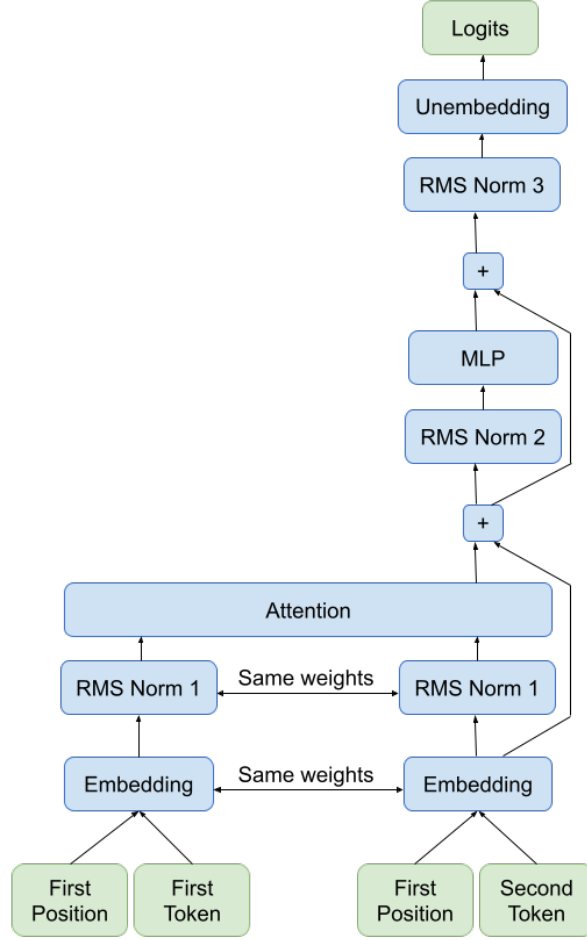


Figure 1: The full toy transformer model, with all the different parts present.

- **None:** There is no attention head, and no positional embedding. Instead there is two different token embeddings, one for each position. These are simply added together to make the first residual stream activation.
- **Uniform:** We remove the attention pattern $\text{softmax}(QK^\top)$ and replace it with a uniform $\frac{1}{2}$.
- **Full:** Just normal attention.

3.2 MLP

There are a number of variants regarding the MLP. Firstly the MLP can either be present or be missing. Secondly if there is an MLP layer, each of the following can be varied

- **Activation Function** can be either GELU or ReLU
- **Bias** can exist or not.³
- **Residual connection** around the MLP can exist or not.

³If the bias is present that means both the linear readout and the linear projection from the ReLU or GELU neurons, have bias. (Making them actually not linear functions but affine functions, in strict mathematical terminology.) If there is no bias, this means neither of these have bias. All other linear connections in the rest of the network (e.g. embeddings, etc) are always bias free.

3.3 Norms

The norms can also be turned on and off. Each of the norms for the readin to the attention and MLP only exist if both that part of the network is present (Uniform or Full for the attention), and Norms are turned on. The last norm, just before the unembedding only depends on the norm setting, and are there if norms are turned on and not there if norms are turned off.

4 First experiment and result: What parts of the network matters?

We trained all different versions of the toy model, to see how many facts each of them could learn. There are some patterns, but unfortunately for most of them, we can't separate what is due to expressability of the model and what is due to learnability.

We say that a model has "learned a fact" if: When the model is given the first two tokens of this sequence, it correctly predicts the correct first token. And by "correctly predicts" we mean that argmaxing over the logits locates the correct output token.

To find out the maximum number of facts a model can learn, we performed a binary search over number of facts, to find the highest number of facts such that the model learned all of them.

For each number of facts we trained 11 models in parallel, with the exact same facts, but different random initialised weights. We used three different success criteria "Any", "Most" and "All", meaning that we said the model succeeded at learning all the facts, if it succeeded in any, most or all of the 11 trials.

We then ran each of these experiments 4 times, to check for stability. The "Any" setting had highest stability (similar max number of fact over all 4 duplicate experiments), and "All" had the worst stability.⁴

All the results for all the experiments are shown in table 1.

4.1 Norms + No Residual around MLP + No Bias + ReLU = Bad

This combination is bad for some reason. We don't know why.

4.2 No Attention > Uniform Attention > Full Attention

It's not surprising that no attention does the best, since the dual embedding that we use to replace attention is (arguably) more powerful. Because attention is non-linear, and the dual embedding is linear, there are things that the attention can express that the dual embedding can't. But on the other hand, the dual embedding gets to encode the input for each token position entirely separately, which gives the network more freedom. Additionally, the no attention setup should be easier to train, since it's simpler.

More surprising is that uniform attention is outperforming full attention, given that full attention is strictly more powerful. Therefore this has to be because of ease of training. This interpretation is also supported by the fact that the number of facts these networks can learn is very variable. We can see this in the number of outliers in table ?? and also how much number of facts drop from Any to Majority to All.

⁴It's not surprising that "All" had bad stability, since it only takes one bad run to throw off the entire batch. But it was not a priori obvious to us that "Any" would be more stable than "Most".

4.3 MLP

Removing the MLP approximaly cutts the numner of leanable facts in half.

4.4 Norms

Norms are generally usfull for learning more facts, with one exception. If the recidual stream around the MLP is turned off, and the MLP bias is turned of, and the activation function is GELU, then and only then, does the norm make the network performe worse. This is true for all trhee settings of the attention.

If ther is no MLP the norm inceases the number of leanable facts with 79% - 191.7%⁵

In the rest of the settings, having norms is increasing the number of learnable facts with 2.2% - 42.2%⁶

4.5 Recidual Connection around the MLP

Adding this recidual connection increases the number of leanable facts with 3.1% - 34%⁷

The one outlier is the case Norm + No Bias + GeLU, where adding a Norm makes a much larger diffrence, becasue of the extra bad synergy of Norms + No Recidual around MLP + No Bias

4.6 Activation Function

GELU is typically better than ReLU. The diffrence is typically small -0.2% to +12.7% from chaning from ReLU to GELU.

Again the exception is Norm + No Recidual around the MLP + No Bias, where swiching from ReLU to GELU boosts the number of leanable facts with 58% - 79%.

5 Challenge / Benchmark for understanding

Can you or me, write down weights for the memory toy model, eitgher by hand or some algorithm that isn't gradient decent, such that our resulting model match the performance of the leaned model?

This chalance is benchmark for how well we understand how the model store the facts. There are two reasons why this is a usefull frameing.

- If we understand how the facts are embedded, we should be able to replicate this, whitout gadient decaent.
- Fhinkging about "How would I do this?" can be a usefull framing for mech-interp.

We think that our current best attempt (which will be pressented soon) is some non-zero progress on this challange, but there are still far to go. We encurrage all readers go give it a try.

5.1 Model architecture

Firstly, we're not trying (yet) to produce weights for the full tranformer model, but instead aiming to find functional weights for the simplified toy model shown in figure 2

⁵Based on data from "Any" the numbers are similar for "Majority" and larger for "All"

⁶Same as last footnote

⁷Based on data from "Any" the numbers are typically larger for "Majority" and larger for "All"

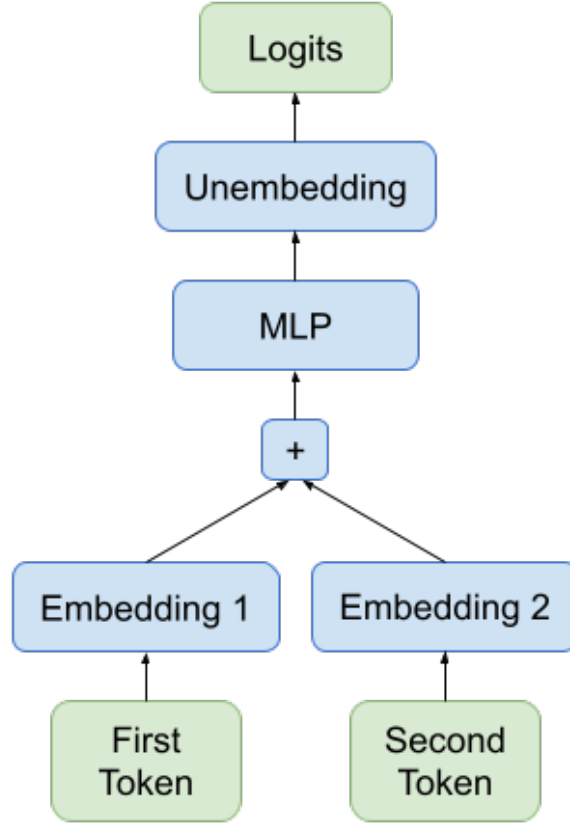


Figure 2: A simplified version of the toy model. The MLP is present but everything else (attention, norms, residual connection around the MLP) is turned off.

Secondly, the version shown in 2 has unnecessarily many weights, which is a legacy from being a cut-down version of the full version shown in 1. Because the MLP is sandwiched between two linear operations, we can skip the weight matrices of the MLP.

We can simplify it further down to this: See figure 3

If you want to give the challenge a go, feel free to use this architecture, or any other that you find easier to work with. The final goal is to be able to write down functional weights for the full transformer model, but we think it's ok to start with a simpler case.

5.2 Our attempt

Our algorithm has three steps

- Assign S number of ReLU neurons to each label. This means that each neuron will be assigned to several labels. We try to generate an assignment that achieves both: Each neuron should have approximately the same labels assigned to it as any other neuron; The maximum neuron overlap between any pair of labels, should be as small as possible.
- Choose the embedding weights such that each ReLU neuron assigned to label l will output zero for all facts with label l .
- Assign a large negative weight from ReLU neurons assigned to l to the logit for l . Assign small positive weights everywhere else.

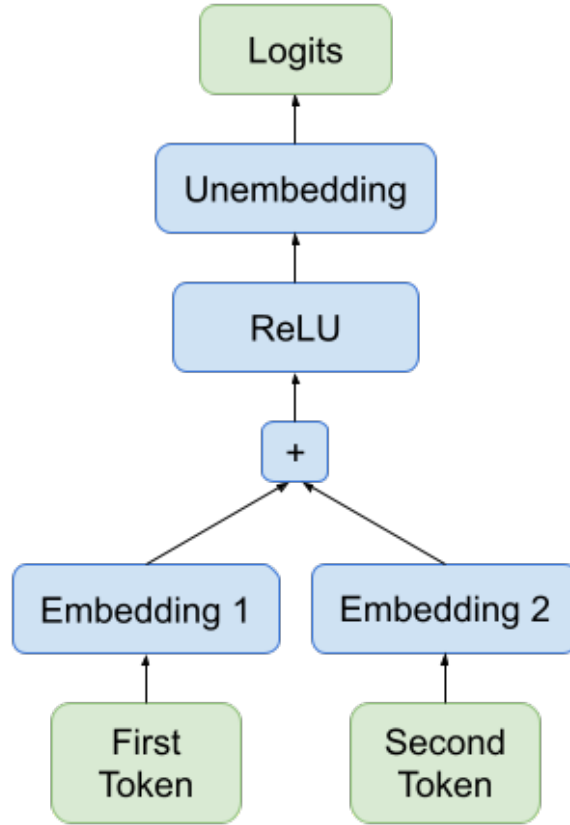


Figure 3: This model is equivalent to the toy model configuration with settings Attn=None, FF=ON, Norm=OFF, Res=OFF, Bias=OFF, Act=ReLU.

5.2.1 Assigning neurons to labels

5.2.2 Embedding weights

5.3 Unembedding weights

Attn	FF	Norm	Res	Bias	Act	Any of 11				Majority of 11				All of 11			
						(1)	(2)	(3)	(4)	(1)	(2)	(3)	(4)	(1)	(2)	(3)	(4)
None	✓	✗	✗	✗	GELU	776	760	784	896	688	672	704	704	640	560	600	560
None	✓	✗	✗	✗	ReLU	664	688	736	816	568	568	504	576	416	<u>184</u>	<u>448</u>	288
None	✓	✗	✗	✓	GELU	864	848	896	904	776	800	784	792	680	680	680	720
None	✓	✗	✗	✓	ReLU	960	824	872	872	680	720	688	712	488	472	528	496
None	✓	✗	✓	✗	GELU	992	984	984	1008	952	960	952	960	920	928	912	928
None	✓	✗	✓	✗	ReLU	976	960	968	976	928	920	920	936	912	888	912	896
None	✓	✗	✓	✓	GELU	1000	1008	992	1008	952	968	960	960	920	920	920	920
None	✓	✗	✓	✓	ReLU	984	976	1000	976	936	928	944	920	904	904	928	912
None	✓	✓	✗	✗	GELU	888	888	920	960	824	856	832	824	792	792	776	800
None	✓	✓	✗	✗	ReLU	472	560	552	576	328	328	<u>248</u>	344	<u>256</u>	168	<u>88</u>	168
None	✓	✓	✗	✓	GELU	1024	1024	1024	1024	1024	1024	1024	1024	984	952	1000	984
None	✓	✓	✗	✓	ReLU	1024	1024	1024	1024	880	888	976	920	528	480	536	536
None	✓	✓	✓	✗	GELU	1024	1024	1024	1024	1000	1016	1024	1024	984	992	984	952
None	✓	✓	✓	✗	ReLU	1024	1024	1024	1024	1016	1024	1024	1024	1008	984	992	992
None	✓	✓	✓	✓	GELU	1024	1024	1024	1024	1024	1024	1024	1024	1024	1024	1000	1024
None	✓	✓	✓	✓	ReLU	1024	1024	1024	1024	1024	1024	1024	1024	1008	1024	1024	1008
None	✗	✗	N/A	N/A	N/A	208	208	208	208	208	208	208	208	208	208	208	208
None	✗	✓	N/A	N/A	N/A	576	576	576	552	536	544	528	544	496	480	472	504
Unif	✓	✗	✗	✗	GELU	696	680	680	688	576	536	552	552	440	424	480	464
Unif	✓	✗	✗	✗	ReLU	640	640	616	672	472	512	480	512	336	376	368	392
Unif	✓	✗	✗	✓	GELU	712	696	720	720	664	640	632	672	528	520	488	520
Unif	✓	✗	✗	✓	ReLU	712	712	704	720	640	640	584	640	448	440	480	432
Unif	✓	✗	✓	✗	GELU	832	824	824	816	792	792	800	792	768	720	744	760
Unif	✓	✗	✓	✗	ReLU	816	808	816	800	760	776	784	784	736	744	728	<u>504</u>
Unif	✓	✗	✓	✓	GELU	824	832	840	840	800	808	800	808	728	760	776	752
Unif	✓	✗	✓	✓	ReLU	824	792	816	800	784	776	784	784	760	768	768	760
Unif	✓	✓	✗	✗	GELU	800	800	776	784	744	736	744	736	696	696	696	704
Unif	✓	✓	✗	✗	ReLU	496	504	528	440	280	296	312	248	104	144	104	<u>160</u>
Unif	✓	✓	✗	✓	GELU	880	904	888	888	848	848	848	840	776	760	776	768
Unif	✓	✓	✗	✓	ReLU	920	888	896	880	768	728	768	768	240	248	248	<u>448</u>
Unif	✓	✓	✓	✗	GELU	880	896	912	920	856	864	848	856	832	816	800	792
Unif	✓	✓	✓	✗	ReLU	872	880	904	872	856	840	848	856	800	792	800	808
Unif	✓	✓	✓	✓	GELU	944	936	960	960	912	904	904	896	856	864	880	856
Unif	✓	✓	✓	✓	ReLU	912	944	936	960	896	888	888	872	816	832	832	832
Unif	✗	✗	N/A	N/A	N/A	208	208	208	208	200	192	208	192	192	192	192	192
Unif	✗	✓	N/A	N/A	N/A	416	408	408	408	376	376	376	376	312	304	328	336
Full	✓	✗	✗	✗	GELU	536	584	504	600	<u>304</u>	392	384	384	<u>136</u>	120	<u>80</u>	88
Full	✓	✗	✗	✗	ReLU	456	576	488	584	312	352	272	312	<u>112</u>	<u>64</u>	<u>128</u>	<u>48</u>
Full	✓	✗	✗	✓	GELU	600	632	696	616	336	384	312	352	<u>144</u>	<u>152</u>	<u>24</u>	<u>48</u>
Full	✓	✗	✗	✓	ReLU	544	656	536	640	280	272	368	360	<u>144</u>	<u>16</u>	72	56
Full	✓	✗	✓	✗	GELU	688	664	656	640	344	328	384	<u>248</u>	<u>56</u>	<u>128</u>	136	<u>40</u>
Full	✓	✗	✓	✗	ReLU	<u>488</u>	672	704	688	336	320	248	248	88	<u>112</u>	64	72
Full	✓	✗	✓	✓	GELU	624	664	712	608	264	296	248	328	<u>16</u>	<u>128</u>	<u>96</u>	<u>56</u>
Full	✓	✗	✓	✓	ReLU	648	600	504	592	272	248	<u>176</u>	512	96	<u>64</u>	80	<u>112</u>
Full	✓	✓	✗	✗	GELU	784	768	768	696	648	656	600	616	264	368	232	232
Full	✓	✓	✗	✗	ReLU	440	512	496	472	312	296	320	328	128	<u>192</u>	152	136
Full	✓	✓	✗	✓	GELU	800	896	800	760	600	672	<u>504</u>	704	336	<u>184</u>	376	296
Full	✓	✓	✗	✓	ReLU	800	776	728	832	440	520	576	536	<u>152</u>	232	264	288
Full	✓	✓	✓	✗	GELU	864	776	832	808	632	704	736	704	232	216	<u>320</u>	184
Full	✓	✓	✓	✗	ReLU	776	832	832	832	680	688	728	688	280	240	280	296
Full	✓	✓	✓	✓	GELU	848	816	872	808	720	696	632	704	320	<u>248</u>	384	<u>448</u>
Full	✓	✓	✓	✓	ReLU	832	840	840	792	768	704	640	632	336	<u>208</u>	320	240
Full	✗	✗	N/A	N/A	N/A	208	168	224	208	112	120	88	96	16	16	16	<u>48</u>
Full	✗	✓	N/A	N/A	N/A	384	<u>272</u>	416	384	168	176	<u>136</u>	192	64	48	56	56

Table 1: Maximum facts memorised per architectural configuration ($d_{\text{residual}} = 16$, $d_{\text{ff}} = 16$, CE loss). **Attn** is the token-mixing mechanism: *None* (additive per-position embeddings), *Unif* (uniform/averaging attention, `qk_is_one = True`), or *Full* (learned attention). The *Any* / *Majority* / *All* column groups apply the corresponding criterion (a fact count counts as learned if at least one / more than half / all of the 11 attempts reach perfect accuracy); the four numbered columns within each group are independent repeats. Within each group the row-wise maximum is shown in **bold** and values more than 20% from the group’s median are boxed as outliers. For `ff = False` rows the residual, bias and activation flags do not apply (N/A). Note that 1024 is the dataset ceiling, so configurations reaching it have saturated the data rather than the model.